

Lines 1-84

```
1 import datetime as dt
2 import os
3 import uuid
4
5 from flask import Flask, abort, flash, redirect, render_templa
> te, request, url_for
6 from flask_login import LoginManager, UserMixin, current_user,
> login_required, login_user, logout_user
7 from sqlite3 import IntegrityError
8 from werkzeug.security import check_password_hash, generate_pa
> ssword_hash
9 from werkzeug.utils import secure_filename
10
11 from db import get_db_connection
12
13 # Configuration
14
15 app = Flask(__name__)
16 app.secret_key = os.environ.get("SECRET_KEY", "dev-secret-key"
> )
17
18 BASE_DIR = os.path.dirname(os.path.abspath(__file__))
19 UPLOAD_FOLDER = os.path.join(BASE_DIR, "static", "uploads")
20 app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
21 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
22
23 ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "gif", "webp"}
24 LANGUAGES = ["Italian", "English", "Spanish", "Portuguese", "G
> erman"]
25 WEEKDAYS = [
26     (0, "Monday"),
27     (1, "Tuesday"),
28     (2, "Wednesday"),
29     (3, "Thursday"),
30     (4, "Friday"),
31     (5, "Saturday"),
32     (6, "Sunday"),
33 ]
34 WEEKDAY_NAMES = dict(WEEKDAYS)
35 DURATION_FILTERS = {
36     "short": ("Up to 90 min", 0, 90),
37     "medium": ("91-150 min", 91, 150),
38     "long": ("Over 150 min", 151, 10000),
39 }
40 MIN_TOUR_STOPS = 4
41
42 login_manager = LoginManager()
43 login_manager.init_app(app)
44 login_manager.login_view = "login"
45
46
47 # Authentication
48
49 class User(UserMixin):
50     def __init__(self, id, email, role, first_name, last_name,
> languages=""):
51         self.id = id
52         self.email = email
53         self.role = role
54         self.first_name = first_name
55         self.last_name = last_name
56         self.languages = languages or ""
57
58     @property
59     def spoken_languages(self):
60         return [lang.strip() for lang in self.languages.split(
> ",") if lang.strip()]
61
62
63 @login_manager.user_loader
64 def load_user(user_id):
65     conn = get_db_connection()
66     user_row = conn.execute("SELECT * FROM users WHERE id = ?"
> , (user_id,)).fetchone()
67     conn.close()
68     if user_row:
69         return User(
70             user_row["id"],
71             user_row["email"],
72             user_row["role"],
73             user_row["first_name"],
74             user_row["last_name"],
75             user_row["languages"],
76         )
77     return None
78
79
80 @app.context_processor
81 def inject_constants():
82     return {
83         "LANGUAGES": LANGUAGES,
```

Lines 85-166

```
85     "WEEKDAYS": WEEKDAYS,
86     "WEEKDAY_NAMES": WEEKDAY_NAMES,
87     "DURATION_FILTERS": DURATION_FILTERS,
88     "MIN_TOUR_STOPS": MIN_TOUR_STOPS,
89 }
90
91 # Validation
92
93 def normalize_email(email):
94     return (email or "").strip().lower()
95
96
97 def allowed_file(filename):
98     return "." in filename and filename.rsplit(".", 1)[1].lowe
> r() in ALLOWED_EXTENSIONS
99
100
101 def validate_file(file):
102     return file and file.filename and allowed_file(file.filena
> me)
103
104
105 def save_uploaded_file(file, prefix):
106     if not validate_file(file):
107         raise ValueError("Invalid file. Use png, jpg, jpeg, gi
> f or webp.")
108     original = secure_filename(file.filename)
109     ext = original.rsplit(".", 1)[1].lower()
110     filename = f"{prefix}-{uuid.uuid4().hex}.{ext}"
111     file.save(os.path.join(app.config["UPLOAD_FOLDER"], filena
> me))
112     return f"/static/uploads/{filename}"
113
114
115 def delete_uploaded_file(path):
116     if not path or not path.startswith("/static/uploads/"):
117         return
118     upload_root = os.path.abspath(app.config["UPLOAD_FOLDER"])
119     filename = secure_filename(os.path.basename(path))
120     file_path = os.path.abspath(os.path.join(upload_root, file
> name))
121     if os.path.commonpath([upload_root, file_path]) != upload_
> root:
122         return
123     if os.path.exists(file_path):
124         os.remove(file_path)
125
126
127 def parse_positive_int(value, label, min_value=1, max_value=No
> ne):
128     try:
129         parsed = int(value)
130     except (TypeError, ValueError):
131         raise ValueError(f"{label} must be a number.")
132     if parsed < min_value:
133         raise ValueError(f"{label} must be at least {min_value
> }.")
134     if max_value is not None and parsed > max_value:
135         raise ValueError(f"{label} cannot be greater than {max
> _value}.")
136     return parsed
137
138
139 def parse_time_to_minutes(value):
140     try:
141         parsed = dt.time.fromisoformat(value)
142     except (TypeError, ValueError):
143         raise ValueError("Invalid time.")
144     return parsed.hour * 60 + parsed.minute
145
146
147 def minutes_to_time_label(total_minutes):
148     hours = (total_minutes // 60) % 24
149     minutes = total_minutes % 60
150     return f"{hours:02d}:{minutes:02d}"
151
152
153 def time_range_label(start_time, duration_mins):
154     start_minutes = parse_time_to_minutes(start_time)
155     end_minutes = start_minutes + duration_mins
156     return f"{minutes_to_time_label(start_minutes)} to {minute
> s_to_time_label(end_minutes)}"
157
158
159 def ranges_overlap(start_a, duration_a, start_b, duration_b):
160     end_a = start_a + duration_a
161     end_b = start_b + duration_b
162     return start_a < end_b and start_b < end_a
163
164
165 def parse_iso_date(value):
```

Lines 167-246

```
167     try:
168         return dt.date.fromisoformat(value)
169     except (TypeError, ValueError):
170         raise ValueError("Invalid date.")
171
172
173 def tour_datetime(tour_date, start_time):
174     return dt.datetime.combine(tour_date, dt.time.fromisoformat(start_time))
175
176
177 def split_names(value):
178     raw = (value or "").replace("\n", ",")
179     return [name.strip() for name in raw.split(",") if name.strip()]
180
181
182 def has_first_and_last_name(value):
183     return len([part for part in value.split() if part.strip()]) >= 2
184
185
186 def safe_redirect(default_endpoint="index"):
187     next_url = request.args.get("next") or request.form.get("next")
188     if next_url and next_url.startswith("/") and not next_url.startswith("//"):
189         return redirect(next_url)
190     return redirect(url_for(default_endpoint))
191
192
193 def require_role(role):
194     if not current_user.is_authenticated:
195         return redirect(url_for("login", next=request.path))
196     if current_user.role != role:
197         flash("This account does not have permission to perform this action.", "warning")
198         return redirect(url_for("index"))
199     return None
200
201
202 # Tour data
203
204 def get_schedules(conn, tour_id):
205     return conn.execute(
206         "SELECT * FROM tour_schedule WHERE tour_id = ? ORDER BY weekday",
207         (tour_id,),
208     ).fetchall()
209
210
211 def get_schedule_for_date(conn, tour_id, tour_date):
212     return conn.execute(
213         """
214         SELECT * FROM tour_schedule
215         WHERE tour_id = ? AND weekday = ?
216         """,
217         (tour_id, tour_date.weekday()),
218     ).fetchone()
219
220
221 def format_schedule(schedules):
222     return ", ".join(f"{WEEKDAY_NAMES[row['weekday']]}" for row in schedules)
223
224
225 def get_stops(conn, tour_id):
226     return conn.execute(
227         "SELECT * FROM tour_stops WHERE tour_id = ? ORDER BY position",
228         (tour_id,),
229     ).fetchall()
230
231
232 def get_photos(conn, tour_id):
233     return conn.execute(
234         "SELECT * FROM tour_photos WHERE tour_id = ? ORDER BY position",
235         (tour_id,),
236     ).fetchall()
237
238
239 def primary_photo(conn, tour_id):
240     photo = conn.execute(
241         "SELECT path FROM tour_photos WHERE tour_id = ? ORDER BY position LIMIT 1",
242         (tour_id,),
243     ).fetchone()
244     return photo["path"] if photo else "/static/assets/img/fav-icon.png"
245
246
```

Lines 247-330

```
247 def tour_like_count(conn, tour_id):
248     row = conn.execute(
249         "SELECT COUNT(*) AS total FROM tour_likes WHERE tour_id = ?",
250         (tour_id,),
251     ).fetchone()
252     return row["total"]
253
254
255 def tour_comment_count(conn, tour_id):
256     row = conn.execute(
257         "SELECT COUNT(*) AS total FROM comments WHERE tour_id = ?",
258         (tour_id,),
259     ).fetchone()
260     return row["total"]
261
262
263 def current_user_liked(conn, tour_id):
264     if not current_user.is_authenticated:
265         return False
266     row = conn.execute(
267         "SELECT 1 FROM tour_likes WHERE tour_id = ? AND user_id = ?",
268         (tour_id, current_user.id),
269     ).fetchone()
270     return row is not None
271
272
273 def get_tour_row(conn, tour_id):
274     return conn.execute(
275         """
276         SELECT tours.*, users.first_name AS guide_first_name,
277                users.last_name AS guide_last_name
278         FROM tours
279         JOIN users ON users.id = tours.guide_id
280         WHERE tours.id = ?
281         """,
282         (tour_id,),
283     ).fetchone()
284
285
286 # Availability
287
288 def active_reserved_places(conn, tour_id, tour_date):
289     row = conn.execute(
290         """
291         SELECT COALESCE(SUM(num_people), 0) AS total
292         FROM reservations
293         WHERE tour_id = ? AND tour_date = ? AND status = 'booked'
294         """,
295         (tour_id, tour_date.isoformat()),
296     ).fetchone()
297     return row["total"]
298
299
300 def available_places(conn, tour, tour_date):
301     return tour["max_participants"] - active_reserved_places(conn, tour["id"], tour_date)
302
303
304 def tour_has_active_reservations(conn, tour_id):
305     row = conn.execute(
306         """
307         SELECT COUNT(*) AS total
308         FROM reservations
309         WHERE tour_id = ? AND status = 'booked'
310         """,
311         (tour_id,),
312     ).fetchone()
313     return row["total"] > 0
314
315
316 def guide_has_overlap(conn, guide_id, weekday, start_time, duration_mins, exclude_tour_id=None):
317     sql = """
318     SELECT tours.id, tours.title, tours.duration_mins, tour_schedule.start_time
319     FROM tours
320     JOIN tour_schedule ON tour_schedule.tour_id = tours.id
321     WHERE tours.guide_id = ? AND tour_schedule.weekday = ?
322     """
323     params = [guide_id, weekday]
324     if exclude_tour_id is not None:
325         sql += " AND tours.id <> ?"
326         params.append(exclude_tour_id)
327
328     start_minutes = parse_time_to_minutes(start_time)
329     for row in conn.execute(sql, params).fetchall():
330         other_start = parse_time_to_minutes(row["start_time"])
```

Lines 331-412

```
331         if ranges_overlap(start_minutes, duration_mins, other_
332 > start, row["duration_mins"]):
333             return row
334         return None
335
336 def participant_has_overlap(conn, user_id, tour_date, start_ti
337 me, duration_mins):
338     rows = conn.execute(
339         """
340         SELECT reservations.id, tours.title, tours.duration_mi
341 ns, tour_schedule.start_time
342 FROM reservations
343 JOIN tours ON tours.id = reservations.tour_id
344 JOIN tour_schedule
345 ON tour_schedule.tour_id = tours.id
346 AND tour_schedule.weekday = ?
347 WHERE reservations.user_id = ?
348 AND reservations.tour_date = ?
349 AND reservations.status = 'booked'
350 """,
351         (tour_date.weekday(), user_id, tour_date.isoformat()),
352     ).fetchall()
353     start_minutes = parse_time_to_minutes(start_time)
354     for row in rows:
355         other_start = parse_time_to_minutes(row["start_time"])
356         if ranges_overlap(start_minutes, duration_mins, other_
357 > start, row["duration_mins"]):
358             return row
359         return None
360
361 def upcoming_dates(conn, tour, horizon_days=60):
362     today = dt.date.today()
363     now = dt.datetime.now()
364     schedules = {row["weekday"]: row for row in get_schedules(
365 > conn, tour["id"])}
366     dates = []
367     for offset in range(horizon_days + 1):
368         candidate = today + dt.timedelta(days=offset)
369         schedule = schedules.get(candidate.weekday())
370         if not schedule:
371             continue
372         start_dt = tour.datetime(candidate, schedule["start_ti
373 > me"])
374         if start_dt <= now:
375             continue
376         seats_left = available_places(conn, tour, candidate)
377         dates.append(
378             {
379                 "date": candidate,
380                 "date_iso": candidate.isoformat(),
381                 "weekday": WEEKDAY_NAMES[candidate.weekday()],
382                 "start_time": schedule["start_time"],
383                 "seats_left": seats_left,
384             }
385         )
386     return dates
387
388 def enrich_tour(conn, row, selected_date=None):
389     tour = dict(row)
390     tour["schedule"] = get_schedules(conn, tour["id"])
391     tour["schedule_label"] = format_schedule(tour["schedule"])
392     tour["primary_photo"] = primary_photo(conn, tour["id"])
393     tour["can_edit"] = not tour_has_active_reservations(conn,
394 > tour["id"])
395     tour["like_count"] = tour_like_count(conn, tour["id"])
396     tour["comment_count"] = tour_comment_count(conn, tour["id"
397 > ])
398     tour["is_liked"] = current_user_liked(conn, tour["id"])
399     if selected_date:
400         schedule = get_schedule_for_date(conn, tour["id"], sel
401 > ected_date)
402         tour["selected_seats_left"] = available_places(conn, t
403 > our, selected_date) if schedule else None
404     return tour
405
406 def filtered_tours(conn, filters):
407     rows = conn.execute(
408         """
409         SELECT tours.*, users.first_name AS guide_first_name,
410         users.last_name AS guide_last_name
411 FROM tours
412 JOIN users ON users.id = tours.guide_id
413 ORDER BY tours.created_at DESC, tours.id DESC
414 """)
415     .fetchall()
416     selected_date = None
417     if filters.get("date"):
```

Lines 413-490

```
413         selected_date = parse_iso_date(filters["date"])
414
415     results = []
416     for row in rows:
417         tour = enrich_tour(conn, row, selected_date)
418         q = (filters.get("q") or "").strip().lower()
419         if q and q not in tour["title"].lower() and q not in t
420 our["theme"].lower():
421             continue
422         if filters.get("language") and tour["language"] != fil
423 ters["language"]:
424             continue
425         duration_key = filters.get("duration")
426         if duration_key:
427             _, min_duration, max_duration = DURATION_FILTERS[d
428 uration_key]
429             if not (min_duration <= tour["duration_mins"] <= m
430 ax_duration):
431                 continue
432         if selected_date and not get_schedule_for_date(conn, t
433 our["id"], selected_date):
434             continue
435         results.append(tour)
436     return results
437
438 # Tour forms
439
440 def parse_schedule_form():
441     schedules = []
442     errors = []
443     for weekday, label in WEEKDAYS:
444         if request.form.get(f"day_{weekday}"):
445             start_time = (request.form.get(f"time_{weekday}")
446 or "").strip()
447             if not start_time:
448                 errors.append(f"Enter a time for {label}.")
449                 continue
450             try:
451                 parse_time_to_minutes(start_time)
452             except ValueError:
453                 errors.append(f"Invalid time for {label}.")
454             continue
455             schedules.append({"weekday": weekday, "start_time"
456 : start_time})
457         if not schedules:
458             errors.append("Select at least one weekday.")
459     return schedules, errors
460
461 def parse_tour_form(conn, exclude_tour_id=None):
462     errors = []
463     title = (request.form.get("title") or "").strip()
464     theme = (request.form.get("theme") or "").strip()
465     meeting_point = (request.form.get("meeting_point") or "").
466 strip()
467     description = (request.form.get("description") or "").stri
468 p()
469     language = (request.form.get("language") or "").strip()
470     stops = split_names(request.form.get("stops"))
471
472     if len(title) < 4:
473         errors.append("The title must contain at least 4 chara
474 cters.")
475     if len(theme) < 3:
476         errors.append("Enter a tour theme.")
477     if len(meeting_point) < 4:
478         errors.append("Enter a clear meeting point.")
479     if len(description) < 30:
480         errors.append("The description must contain at least 3
481 0 characters.")
482     if language not in current_user.spoken_languages:
483         errors.append("The tour language must be one of the gu
484 ide's spoken languages.")
485     if len(stops) < MIN_TOUR_STOPS:
486         errors.append(f"Enter at least {MIN_TOUR_STOPS} stops.
487 ")
488
489     try:
490         duration_mins = parse_positive_int(request.form.get("d
491 uration_mins"), "Duration", 30, 360)
492     except ValueError as exc:
493         duration_mins = None
494         errors.append(str(exc))
495
496     try:
497         max_participants = parse_positive_int(
498             request.form.get("max_participants"),
499             "Maximum number of participants",
500             1,
501             40,
502         )
```

Lines 491-573

```

491     except ValueError as exc:
492         max_participants = None
493         errors.append(str(exc))
494
495     schedules, schedule_errors = parse_schedule_form()
496     errors.extend(schedule_errors)
497
498     if duration_mins:
499         for schedule in schedules:
500             overlap = guide_has_overlap(
501                 conn,
502                 current_user.id,
503                 schedule["weekday"],
504                 schedule["start_time"],
505                 duration_mins,
506                 exclude_tour_id,
507             )
508             if overlap:
509                 errors.append(
510                     f"Schedule overlap: {WEEKDAY_NAMES[schedule['weekday']]} "
511                     f"{schedule['start_time']} overlaps with "
512                     f"{overlap['title']}"
513                     f"from {time_range_label(overlap['start_time'], overlap['duration_mins'])}."
514                 )
515             return {
516                 "title": title,
517                 "theme": theme,
518                 "meeting_point": meeting_point,
519                 "duration_mins": duration_mins,
520                 "language": language,
521                 "max_participants": max_participants,
522                 "description": description,
523                 "stops": stops,
524                 "schedules": schedules,
525             }, errors
526
527
528 def get_form_photo_files():
529     return [file for file in request.files.getlist("photos") if file and file.filename]
530
531
532 def validate_photo_files(files, required):
533     if required and len(files) < 5:
534         return "Upload at least 5 promotional photos."
535     for file in files:
536         if not validate_file(file):
537             return "Photos must be png, jpg, jpeg, gif or webp."
538     return None
539
540
541 def parse_photo_ids(values):
542     photo_ids = set()
543     for value in values:
544         try:
545             photo_id = int(value)
546         except (TypeError, ValueError):
547             raise ValueError("Invalid photo selection.")
548         if photo_id <= 0:
549             raise ValueError("Invalid photo selection.")
550         photo_ids.add(photo_id)
551     return photo_ids
552
553
554 def reindex_tour_photos(conn, tour_id):
555     rows = conn.execute(
556         "SELECT id FROM tour_photos WHERE tour_id = ? ORDER BY "
557         "position, id",
558         (tour_id,),
559     ).fetchall()
560     for position, row in enumerate(rows, start=1):
561         conn.execute(
562             "UPDATE tour_photos SET position = ? WHERE id = ?",
563             (position, row["id"]),
564         )
565
566 def insert_tour_details(conn, tour_id, data, photo_files):
567     for schedule in data["schedules"]:
568         conn.execute(
569             "INSERT INTO tour_schedule (tour_id, weekday, start_time) VALUES (?, ?, ?)",
570             (tour_id, schedule["weekday"], schedule["start_time"]),
571         )
572     for position, stop in enumerate(data["stops"], start=1):
573         conn.execute(

```

Lines 574-655

```

574         "INSERT INTO tour_stops (tour_id, name, position) "
575         "VALUES (?, ?, ?)",
576         (tour_id, stop, position),
577     )
578     for position, file in enumerate(photo_files, start=1):
579         path = save_uploaded_file(file, f"tour-{tour_id}-{position}")
580         conn.execute(
581             "INSERT INTO tour_photos (tour_id, path, position) "
582             "VALUES (?, ?, ?)",
583             (tour_id, path, position),
584         )
585
586 # Public pages
587
588 @app.route("/")
589 def index():
590     conn = get_db_connection()
591     tours = sorted(
592         filtered_tours(conn, {}),
593         key=lambda item: (item["like_count"], item["comment_count"], item["id"]),
594         reverse=True,
595     )[:3]
596     conn.close()
597     return render_template("index.html", tours=tours)
598
599
600 @app.route("/tours")
601 def tours():
602     filters = {
603         "q": request.args.get("q", ""),
604         "date": request.args.get("date", ""),
605         "duration": request.args.get("duration", ""),
606         "language": request.args.get("language", ""),
607     }
608     if filters["duration"] and filters["duration"] not in DURATION_FILTERS:
609         filters["duration"] = ""
610     if filters["language"] and filters["language"] not in LANGUAGES:
611         filters["language"] = ""
612
613     conn = get_db_connection()
614     try:
615         tour_list = filtered_tours(conn, filters)
616     except ValueError:
617         flash("The date filter is not valid.", "warning")
618         filters["date"] = ""
619         tour_list = filtered_tours(conn, filters)
620     conn.close()
621     return render_template("tours.html", tours=tour_list, filters=filters)
622
623 # Account routes
624
625 @app.route("/login", methods=["GET", "POST"])
626 def login():
627     if request.method == "POST":
628         email = normalize_email(request.form.get("email"))
629         password = request.form.get("password") or ""
630         role = request.form.get("role")
631
632         if role not in {"guide", "participant"}:
633             flash("Select an account type.", "danger")
634             return render_template("login.html")
635
636         conn = get_db_connection()
637         user_row = conn.execute(
638             "SELECT * FROM users WHERE email = ? AND role = ?",
639             (email, role),
640         ).fetchone()
641         conn.close()
642
643         if user_row and check_password_hash(user_row["password"], password):
644             user_obj = User(
645                 user_row["id"],
646                 user_row["email"],
647                 user_row["role"],
648                 user_row["first_name"],
649                 user_row["last_name"],
650                 user_row["languages"],
651             )
652             login_user(user_obj)
653             return safe_redirect()
654         flash("Wrong credentials for the selected account type.", "danger")
655         return render_template("login.html")

```

Lines 656-737

```

656
657
658 @app.route("/register", methods=["GET", "POST"])
659 def register():
660     if request.method == "POST":
661         first_name = (request.form.get("first_name") or "").stri
662         > rip()
663         last_name = (request.form.get("last_name") or "").stri
664         > p()
665         email = normalize_email(request.form.get("email"))
666         password = request.form.get("password") or ""
667         role = request.form.get("role")
668         selected_languages = request.form.getlist("languages")
669         errors = []
670
671         if len(first_name) < 2:
672             errors.append("Enter a valid first name.")
673         if len(last_name) < 2:
674             errors.append("Enter a valid last name.")
675         if "@" not in email or "." not in email:
676             errors.append("Enter a valid email.")
677         if len(password) < 6:
678             errors.append("The password must have at least 6 c
679             > haracters.")
680         if role not in {"guide", "participant"}:
681             errors.append("Select a valid account type.")
682
683         languages = ""
684         if role == "guide":
685             valid_languages = [lang for lang in selected_langu
686             > ages if lang in LANGUAGES]
687             if not valid_languages:
688                 errors.append("A guide must select at least on
689                 > e language.")
690             languages = ",".join(valid_languages)
691
692         if errors:
693             for error in errors:
694                 flash(error, "danger")
695             return render_template("register.html")
696
697         conn = get_db_connection()
698         try:
699             conn.execute(
700                 """
701                 INSERT INTO users (first_name, last_name, email,
702                 > password, role, languages)
703                 VALUES (?, ?, ?, ?, ?, ?)
704                 """
705                 (
706                     first_name,
707                     last_name,
708                     email,
709                     generate_password_hash(password, method="p
710                     > bkdf2:sha256"),
711                     role,
712                     languages,
713                 ),
714             )
715             conn.commit()
716             flash("Registration completed. You can now sign in
717             > .", "success")
718             return redirect(url_for("login"))
719         except IntegrityError:
720             flash("An account with this email already exists."
721             > , "danger")
722         finally:
723             conn.close()
724         return render_template("register.html")
725
726
727 @app.route("/logout")
728 @login_required
729 def logout():
730     logout_user()
731     return redirect(url_for("index"))
732
733
734 # Guide tours
735
736 @app.route("/create-tour", methods=["GET", "POST"])
737 @login_required
738 def create_tour():
739     denied = require_role("guide")
740     if denied:
741         return denied
742
743     if request.method == "POST":
744         conn = get_db_connection()
745         data, errors = parse_tour_form(conn)
746         photo_files = get_form_photo_files()

```

Lines 738-816

```

738         photo_error = validate_photo_files(photo_files, requir
739         > ed=True)
740         if photo_error:
741             errors.append(photo_error)
742
743         if errors:
744             for error in errors:
745                 flash(error, "danger")
746             conn.close()
747             return render_template("create_tour.html", tour=None,
748             > selected_schedules=[])
749
750         try:
751             cursor = conn.execute(
752                 """
753                 INSERT INTO tours
754                 (guide_id, title, theme, meeting_point, du
755                 > ration_mins,
756                 language, max_participants, description)
757                 VALUES (?, ?, ?, ?, ?, ?, ?, ?)
758                 """
759                 (
760                     current_user.id,
761                     data["title"],
762                     data["theme"],
763                     data["meeting_point"],
764                     data["duration_mins"],
765                     data["language"],
766                     data["max_participants"],
767                     data["description"],
768                 ),
769             )
770             tour_id = cursor.lastrowid
771             insert_tour_details(conn, tour_id, data, photo_fil
772             > es)
773             conn.commit()
774             flash("Tour planned successfully.", "success")
775             return redirect(url_for("tour_detail", id=tour_id)
776             > )
777         except ValueError as exc:
778             conn.rollback()
779             flash(str(exc), "danger")
780         finally:
781             conn.close()
782
783         return render_template("create_tour.html", tour=None, sele
784         > cted_schedules=[])
785
786
787 @app.route("/tour/<int:id>/edit", methods=["GET", "POST"])
788 @login_required
789 def edit_tour(id):
790     denied = require_role("guide")
791     if denied:
792         return denied
793
794     conn = get_db_connection()
795     tour = get_tour_row(conn, id)
796     if not tour:
797         conn.close()
798         abort(404)
799     if tour["guide_id"] != int(current_user.id):
800         conn.close()
801         abort(403)
802     if tour.has_active_reservations(conn, id):
803         conn.close()
804         flash("This tour has active reservations and cannot be
805         > edited.", "warning")
806         return redirect(url_for("tour_detail", id=id))
807
808     selected_schedules = {row["weekday"]: row["start_time"] fo
809     > r row in get_schedules(conn, id)}
810     stops = ",".join(row["name"] for row in get_stops(conn, i
811     > d))
812     current_photos = get_photos(conn, id)
813     tour_data = dict(tour)
814     tour_data["stops_text"] = stops
815
816     if request.method == "POST":
817         data, errors = parse_tour_form(conn, exclude_tour_id=i
818         > d)
819     photo_files = get_form_photo_files()
820     remove_photo_ids = set()
821     photo_error = validate_photo_files(photo_files, requir
822     > ed=False)
823     if photo_error:
824         errors.append(photo_error)
825
826     try:
827         remove_photo_ids = parse_photo_ids(request.form.ge
828         > tlist("remove_photos"))
829         current_photo_ids = {photo["id"] for photo in curr
830         > ent_photos}

```


Lines 817-892

```

817         if remove_photo_ids - current_photo_ids:
818             errors.append("Invalid photo selection.")
819     except ValueError as exc:
820         errors.append(str(exc))
821
822     final_photo_count = len(current_photos) - len(remove_p
823 > hoto_ids) + len(photo_files)
824     if (remove_photo_ids or photo_files) and final_photo_c
825 > ount < 5:
826         errors.append("A tour must keep at least 5 promoti
827 > onal photos.")
828     removed_photo_paths = [
829 > photo["path"] for photo in current_photos if photo
830 > ["id"] in remove_photo_ids
831 > ]
832
833     if errors:
834         for error in errors:
835             flash(error, "danger")
836         conn.close()
837         return render_template(
838             "create_tour.html",
839             tour=tour_data,
840             selected_schedules=selected_schedules,
841             current_photos=current_photos,
842         )
843
844     try:
845         conn.execute(
846             """
847             UPDATE tours
848             SET title = ?, theme = ?, meeting_point = ?, d
849 > uration_mins = ?,
850 > language = ?, max_participants = ?, descri
851 > ption = ?
852             WHERE id = ?
853             """
854             (
855                 data["title"],
856                 data["theme"],
857                 data["meeting_point"],
858                 data["duration_mins"],
859                 data["language"],
860                 data["max_participants"],
861                 data["description"],
862                 id,
863             ),
864         )
865         conn.execute("DELETE FROM tour_schedule WHERE tour
866 > _id = ?", (id,))
867         conn.execute("DELETE FROM tour_stops WHERE tour_id
868 > = ?", (id,))
869         for schedule in data["schedules"]:
870             conn.execute(
871                 "INSERT INTO tour_schedule (tour_id, weekd
872 > ay, start_time) VALUES (?, ?, ?)",
873                 (id, schedule["weekday"], schedule["start_
874 > time"]),
875             )
876         for position, stop in enumerate(data["stops"], sta
877 > rt=1):
878             conn.execute(
879                 "INSERT INTO tour_stops (tour_id, name, po
880 > sition) VALUES (?, ?, ?)",
881                 (id, stop, position),
882             )
883         for photo_id in remove_photo_ids:
884             conn.execute(
885                 "DELETE FROM tour_photos WHERE tour_id = ?
886 > AND id = ?",
887                 (id, photo_id),
888             )
889         if photo_files:
890             row = conn.execute(
891                 "SELECT COALESCE(MAX(position), 0) AS max_
892 > position FROM tour_photos WHERE tour_id = ?",
893                 (id,))
894             row.fetchone()
895             start_position = row["max_position"] + 1
896             for offset, file in enumerate(photo_files):
897                 position = start_position + offset
898                 path = save_uploaded_file(file, f"tour-{id
899 > }-{position}")
900             conn.execute(
901                 "INSERT INTO tour_photos (tour_id, pat
902 > h, position) VALUES (?, ?, ?)",
903                 (id, path, position),
904             )
905         if remove_photo_ids or photo_files:
906             reindex_tour_photos(conn, id)
907         conn.commit()
908         for path in removed_photo_paths:

```

Lines 893-979

```

893         delete_uploaded_file(path)
894         flash("Tour updated.", "success")
895         return redirect(url_for("tour_detail", id=id))
896     except ValueError as exc:
897         conn.rollback()
898         flash(str(exc), "danger")
899     finally:
900         conn.close()
901
902     else:
903         conn.close()
904
905     return render_template(
906         "create_tour.html",
907         tour=tour_data,
908         selected_schedules=selected_schedules,
909         current_photos=current_photos,
910     )
911
912     # Tour actions
913
914 @app.route("/tour/<int:id>")
915 def tour_detail(id):
916     conn = get_db_connection()
917     tour_row = get_tour_row(conn, id)
918     if not tour_row:
919         conn.close()
920         abort(404)
921
922     tour = enrich_tour(conn, tour_row)
923     stops = get_stops(conn, id)
924     photos = get_photos(conn, id)
925     dates = upcoming_dates(conn, tour)
926     comments = conn.execute(
927         """
928         SELECT comments.*, users.first_name, users.last_name,
929 > users.role
930         FROM comments
931         JOIN users ON users.id = comments.user_id
932         WHERE comments.tour_id = ?
933         ORDER BY comments.id DESC
934         """
935         (id,))
936     ).fetchall()
937     conn.close()
938
939     return render_template(
940         "tour_detail.html",
941         tour=tour,
942         stops=stops,
943         photos=photos,
944         upcoming_dates=dates,
945         comments=comments,
946     )
947
948 @app.route("/tour/<int:id>/like", methods=["POST"])
949 def toggle_like(id):
950     if not current_user.is_authenticated:
951         flash("Sign in to like this tour.", "warning")
952         return redirect(url_for("login", next=url_for("tour_de
953 > tail", id=id)))
954
955     conn = get_db_connection()
956     if not get_tour_row(conn, id):
957         conn.close()
958         abort(404)
959
960     existing = conn.execute(
961         "SELECT id FROM tour_likes WHERE tour_id = ? AND user_
962 > id = ?",
963         (id, current_user.id),
964     ).fetchone()
965     if existing:
966         conn.execute("DELETE FROM tour_likes WHERE id = ?", (e
967 > xisting["id"],))
968         flash("Like removed.", "success")
969     else:
970         conn.execute(
971             "INSERT INTO tour_likes (tour_id, user_id) VALUES
972 > (?, ?)",
973             (id, current_user.id),
974         )
975         flash("Like added.", "success")
976         conn.commit()
977         conn.close()
978         return redirect(url_for("tour_detail", id=id))
979
980 @app.route("/tour/<int:id>/book", methods=["POST"])
981 def book_tour(id):
982     if not current_user.is_authenticated:

```

Lines 980-1053

```
980     flash("Sign in as a participant to book.", "warning")
981     return redirect(url_for("login", next=url_for("tour_detail", id=id)))
982 > tail", id=id)))
983     if current_user.role != "participant":
984         flash("Guides cannot book with a guide account.", "warning")
985         return redirect(url_for("tour_detail", id=id))
986     conn = get_db_connection()
987     tour = get_tour_row(conn, id)
988     if not tour:
989         conn.close()
990         abort(404)
991
992     try:
993         tour_date = parse_iso_date(request.form.get("tour_date"))
994     > )))
995         if tour_date < dt.date.today():
996             raise ValueError("You cannot book a past date.")
997         schedule = get_schedule_for_date(conn, id, tour_date)
998         if not schedule:
999             raise ValueError("The selected date is not part of this tour schedule.")
1000         if tour_datetime(tour_date, schedule["start_time"]) <= dt.datetime.now():
1001             raise ValueError("This tour date has already started or ended.")
1002
1003         num_people = parse_positive_int(request.form.get("num_people"), "Number of participants", 1, 4)
1004         names = split_names(request.form.get("extra_names"))
1005         expected_extra = num_people - 1
1006         if len(names) != expected_extra:
1007             raise ValueError(f"Enter exactly {expected_extra} guest full names for this booking.")
1008         invalid_names = [name for name in names if not has_first_and_last_name(name)]
1009         if invalid_names:
1010             raise ValueError("Invalid guest name format: enter first name and last name for each guest.")
1011         seats_left = available_places(conn, tour, tour_date)
1012         if num_people > seats_left:
1013             raise ValueError(f"Not enough seats: {seats_left} left.")
1014
1015         overlap = participant_has_overlap(conn, current_user.id, tour_date, schedule["start_time"], tour["duration_mins"])
1016         if overlap:
1017             raise ValueError(f"You already have an overlapping booking: {overlap['title']} "
1018                             f"from {time_range_label(overlap['start_time'], overlap['duration_mins'])}.")
1019
1020         existing = conn.execute(
1021             """
1022             SELECT * FROM reservations
1023             WHERE user_id = ? AND tour_id = ? AND tour_date = ?
1024             """,
1025             (current_user.id, id, tour_date.isoformat()),
1026         ).fetchone()
1027         if existing and existing["status"] == "booked":
1028             raise ValueError("You have already booked this tour on this date.")
1029         if existing and existing["status"] == "cancelled":
1030             conn.execute(
1031                 """
1032                 UPDATE reservations
1033                 SET num_people = ?, extra_names = ?, status = 'booked', cancelled_at = NULL
1034                 WHERE id = ?
1035                 """,
1036                 (num_people, ".join(names)", existing["id"])
1037             )
1038         else:
1039             conn.execute(
1040                 """
1041                 INSERT INTO reservations (user_id, tour_id, tour_date, num_people, extra_names)
1042                 VALUES (?, ?, ?, ?, ?)
1043             """
1044             )
```

Lines 1054-1134

```
1054         """
1055         (current_user.id, id, tour_date.isoformat(), num_people, ".join(names)),
1056     )
1057     conn.commit()
1058     flash("Booking confirmed.", "success")
1059     except ValueError as exc:
1060         flash(str(exc), "danger")
1061     except IntegrityError:
1062         flash("A booking already exists for this date.", "danger")
1063     finally:
1064         conn.close()
1065     return redirect(url_for("tour_detail", id=id))
1066
1067 @app.route("/tour/<int:id>/comment", methods=["POST"])
1068 def add_comment(id):
1069     if not current_user.is_authenticated:
1070         flash("Sign in to comment.", "warning")
1071     return redirect(url_for("login", next=url_for("tour_detail", id=id)))
1072
1073     conn = get_db_connection()
1074     if not get_tour_row(conn, id):
1075         conn.close()
1076         abort(404)
1077     text = (request.form.get("text") or "").strip()
1078     if len(text) < 2:
1079         flash("The comment is too short.", "danger")
1080     elif len(text) > 600:
1081         flash("The comment cannot exceed 600 characters.", "danger")
1082     else:
1083         conn.execute(
1084             """
1085             INSERT INTO comments (tour_id, user_id, publication_date, text)
1086             VALUES (?, ?, ?, ?)
1087             """,
1088             (id, current_user.id, dt.date.today().isoformat(), text),
1089         )
1090     conn.commit()
1091     flash("Comment posted.", "success")
1092     conn.close()
1093     return redirect(url_for("tour_detail", id=id))
1094
1095 # Profiles
1096
1097 @app.route("/profile")
1098 @login_required
1099 def profile():
1100     if current_user.role == "guide":
1101         return redirect(url_for("guide_profile"))
1102     return redirect(url_for("participant_profile"))
1103
1104 @app.route("/participant/profile")
1105 @login_required
1106 def participant_profile():
1107     denied = require_role("participant")
1108     if denied:
1109         return denied
1110
1111     conn = get_db_connection()
1112     rows = conn.execute(
1113         """
1114         SELECT reservations.*, tours.title, tours.meeting_point, tours.duration_mins,
1115         tours.language, users.first_name AS guide_first_name,
1116         users.last_name AS guide_last_name
1117         FROM reservations
1118         JOIN tours ON tours.id = reservations.tour_id
1119         JOIN users ON users.id = tours.guide_id
1120         WHERE reservations.user_id = ?
1121         ORDER BY reservations.tour_date DESC, reservations.id DESC
1122         """,
1123         (current_user.id,)
1124     ).fetchall()
1125
1126     reservations = []
1127     now = dt.datetime.now()
1128     for row in rows:
1129         item = dict(row)
1130         tour_date = parse_iso_date(row["tour_date"])
1131         schedule = get_schedule_for_date(conn, row["tour_id"], tour_date)
```

Lines 1135-1214

```
1135         item["start_time"] = schedule["start_time"] if schedul
1136     > e else "-
1137         item["can_cancel"] = False
1138         if row["status"] == "booked" and schedule:
1139             start_dt = tour_datetime(tour_date, schedule["star
1140         t_time"])
1141         item["can_cancel"] = start_dt - now >= dt.timedelta
1142         a(hours=24)
1143         reservations.append(item)
1144         conn.close()
1145         return render_template("participant_profile.html", reserva
1146         tions=reservations)
1147
1148 @app.route("/reservation/<int:reservation_id>/cancel", methods
1149 =["POST"])
1150 @login_required
1151 def cancel_reservation(reservation_id):
1152     denied = require_role("participant")
1153     if denied:
1154         return denied
1155
1156     conn = get_db_connection()
1157     reservation = conn.execute(
1158         "SELECT * FROM reservations WHERE id = ? AND user_id =
1159         ?",
1160         (reservation_id, current_user.id),
1161     ).fetchone()
1162     if not reservation:
1163         conn.close()
1164         abort(404)
1165     if reservation["status"] != "booked":
1166         flash("This booking has already been cancelled.", "war
1167         ning")
1168     conn.close()
1169     return redirect(url_for("participant_profile"))
1170
1171     tour_date = parse_iso_date(reservation["tour_date"])
1172     schedule = get_schedule_for_date(conn, reservation["tour_i
1173     d"], tour_date)
1174     if not schedule or tour_datetime(tour_date, schedule["star
1175     t_time"]) - dt.datetime.now() < dt.timedelta(hours=24):
1176         flash("You can cancel only at least 24 hours before th
1177         e tour starts.", "danger")
1178     conn.close()
1179     return redirect(url_for("participant_profile"))
1180
1181     conn.execute(
1182         """
1183         UPDATE reservations
1184         SET status = 'cancelled', cancelled_at = CURRENT_TIMES
1185         WHERE id = ?
1186         """,
1187         (reservation_id,),
1188     )
1189     conn.commit()
1190     conn.close()
1191     flash("Booking cancelled.", "success")
1192     return redirect(url_for("participant_profile"))
1193
1194 @app.route("/guide/profile")
1195 @login_required
1196 def guide_profile():
1197     denied = require_role("guide")
1198     if denied:
1199         return denied
1200
1201     conn = get_db_connection()
1202     rows = conn.execute(
1203         """
1204         SELECT tours.*, users.first_name AS guide_first_name,
1205         users.last_name AS guide_last_name
1206         FROM tours
1207         JOIN users ON users.id = tours.guide_id
1208         WHERE guide_id = ?
1209         ORDER BY tours.created_at DESC, tours.id DESC
1210         """,
1211         (current_user.id,),
1212     ).fetchall()
1213     tours = []
1214     now = dt.datetime.now()
1215     for row in rows:
1216         tour = enrich_tour(conn, row)
1217         groups = conn.execute(
1218             """
1219             SELECT tour_date, COUNT(*) AS reservation_count,
1220             COALESCE(SUM(num_people), 0) AS people_coun
1221             FROM reservations
1222             WHERE tour_id = ? AND status = 'booked'
```

Lines 1215-1291

```
1215         GROUP BY tour_date
1216         ORDER BY tour_date
1217         """,
1218         (tour["id"],),
1219     ).fetchall()
1220     tour["reservation_dates"] = []
1221     for group in groups:
1222         tour_date = parse_iso_date(group["tour_date"])
1223         schedule = get_schedule_for_date(conn, tour["id"],
1224         tour_date)
1225         start_time = schedule["start_time"] if schedule el
1226     se "-"
1227         start_dt = tour_datetime(tour_date, start_time) if
1228         schedule else None
1229         report = conn.execute(
1230             "SELECT * FROM tour_reports WHERE tour_id = ?
1231             AND tour_date = ?",
1232             (tour["id"], group["tour_date"])),
1233         ).fetchone()
1234         reservations = conn.execute(
1235             """
1236             SELECT reservations.*, users.first_name, users
1237             .last_name, users.email
1238             FROM reservations
1239             JOIN users ON users.id = reservations.user_id
1240             WHERE reservations.tour_id = ?
1241             AND reservations.tour_date = ?
1242             AND reservations.status = 'booked'
1243             ORDER BY reservations.id
1244             """,
1245             (tour["id"], group["tour_date"])),
1246         ).fetchall()
1247         tour["reservation_dates"].append(
1248             {
1249                 "date": group["tour_date"],
1250                 "start_time": start_time,
1251                 "reservation_count": group["reservation_co
1252                 unt"],
1253                 "people_count": group["people_count"],
1254                 "reservations": reservations,
1255                 "report": report,
1256                 "can_report": bool(start_dt and start_dt <
1257                 now and not report),
1258             }
1259         )
1260     tours.append(tour)
1261     conn.close()
1262     return render_template("guide_profile.html", tours=tours)
1263
1264 # Reports
1265
1266 @app.route("/guide/report/<int:tour_id>/<tour_date>", methods=
1267 ["POST"])
1268 @login_required
1269 def submit_report(tour_id, tour_date):
1270     denied = require_role("guide")
1271     if denied:
1272         return denied
1273
1274     conn = get_db_connection()
1275     tour = get_tour_row(conn, tour_id)
1276     if not tour:
1277         conn.close()
1278         abort(404)
1279     if tour["guide_id"] != int(current_user.id):
1280         conn.close()
1281         abort(403)
1282
1283     try:
1284         parsed_date = parse_iso_date(tour_date)
1285         schedule = get_schedule_for_date(conn, tour_id, parsed
1286         _date)
1287         if not schedule:
1288             raise ValueError("This date does not belong to the
1289             tour schedule.")
1290         if tour_datetime(parsed_date, schedule["start_time"])
1291         >= dt.datetime.now():
1292             raise ValueError("The report can be submitted only
1293             after the tour has taken place.")
1294
1295         expected = active_reserved_places(conn, tour_id, parse
1296         d_date)
1297         if expected <= 0:
1298             raise ValueError("There are no bookings for this d
1299             ate.")
1300         existing_report = conn.execute(
1301             "SELECT id FROM tour_reports WHERE tour_id = ? AND
1302             tour_date = ?",
1303             (tour_id, tour_date),
1304         ).fetchone()
1305         if existing_report:
```


Lines 1292-1305

```
1292         raise ValueError("The report for this date has alr
> eady been submitted.")
1293     actual = parse_positive_int(
1294         request.form.get("actual_participants"),
1295         "Actual participants",
1296         0,
1297         expected,
1298     )
1299     file = request.files.get("evidence_photo")
1300     if not validate_file(file):
1301         raise ValueError("Upload a valid evidence photo.")
1302     photo_path = save_uploaded_file(file, f"report-{tour_i
> d}-{tour_date}")
1303
1304     conn.execute(
1305         """
```

Lines 1306-1321

```
1306         INSERT INTO tour_reports (tour_id, tour_date, actu
> al_participants, evidence_photo)
1307         VALUES (?, ?, ?, ?)
1308         """
1309         (tour_id, tour_date, actual, photo_path),
1310     )
1311     conn.commit()
1312     flash("Report saved.", "success")
1313 except ValueError as exc:
1314     flash(str(exc), "danger")
1315 finally:
1316     conn.close()
1317     return redirect(url_for("guide_profile"))
1318
1319
1320 if __name__ == "__main__":
1321     app.run(host="0.0.0.0", debug=True)
```